

Punto 3.7 — Joins y navegación en consultas

Contexto del proyecto

Este checkpoint continúa `M03/PROYECTO/3.6`. Se mantiene la separación Domain/Application/Infrastructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Joins y navegación en consultas** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.7
dotnet restore AceriaData.sln
```

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerJoinOrdenesPlanchas`, `ObtenerLeftJoinOrdenesPlanchas`, `ObtenerOrdenesConDetalleJoin`, `ObtenerOrdenesConAleaciones`, `ObtenerSqlJoinExplicito`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infrastructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/JoinsUseCase.cs`

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class JoinsUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public JoinsUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== JOINS Y NAVEGACIÓN ===");
        var inner = _unidad.Ordenes.ObtenerJoinOrdenesPlanchas();
        var left = _unidad.Ordenes.ObtenerLeftJoinOrdenesPlanchas();
        var detalle = _unidad.Ordenes.ObtenerOrdenesConDetalleJoin();
        var aleaciones = _unidad.Ordenes.ObtenerOrdenesConAleaciones();
        if (inner.Count != 5 || left.Count != 6 || detalle.Count != 5 || aleaciones.Count != 5) throw
new InvalidOperationException("Joins inesperados.");
        if (!left.Any(x => x.NumeroOrden == "OF-2024-0004" && x.Peso is null)) throw new
InvalidOperationException("LEFT JOIN no conservó la orden sin planchas.");
        Console.WriteLine($"INNER filas: {inner.Count} | LEFT filas: {left.Count}");
        Console.WriteLine(_unidad.Ordenes.ObtenerSqlJoinExplicito());
    }
}

```

Explicación línea a línea del caso de uso 3.7

Línea 1: `using AceriaData.Application.Interfaces;` → Importa los puertos de Application, en especial IUnidadDeTrabajo e IOrdenRepositorio, que desacoplan el caso de uso de EF Core.

Línea 3: `namespace AceriaData.Application.UseCases;` → Declara el espacio de nombres de la capa a la que pertenece el archivo.

Línea 5: `public sealed class JoinsUseCase` → Declara el caso de uso concreto que coordina la demostración del punto.

Línea 6: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de DbContext.

Línea 8: `public JoinsUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;` → Recibe la unidad de trabajo mediante inyección de dependencias.

Línea 10: `public void Ejecutar()` → Define la operación docente que ejecutará el composition root.

Línea 11: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 12: `Console.WriteLine("=== JOINS Y NAVEGACIÓN ===");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 13: `var inner = _unidad.Ordenes.ObtenerJoinOrdenesPlanchas();` → Ejecuta el INNER JOIN explícito entre órdenes y planchas y conserva una fila por coincidencia.

Línea 14: `var left = _unidad.Ordenes.ObtenerLeftJoinOrdenesPlanchas();` → Ejecuta el patrón de LEFT JOIN con DefaultIfEmpty para conservar también las órdenes sin planchas.

Línea 15: `var detalle = _unidad.Ordenes.ObtenerOrdenesConDetalleJoin();` → Combina órdenes y detalle mediante navegación/proyección para validar una relación de referencia opcional.

Línea 16: `var aleaciones = _unidad.Ordenes.ObtenerOrdenesConAleaciones();` → Proyecta órdenes junto con sus relaciones de aleación para comprobar la navegación a través de la entidad intermedia.

Línea 17: `if (inner.Count != 5 || left.Count != 6 || detalle.Count != 5 || aleaciones.Count != 5) throw new InvalidOperationException("Joins inesperados.");` → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.

Línea 18: `if (!left.Any(x => x.NumeroOrden == "OF-2024-0004" && x.Peso is null)) throw new InvalidOperationException("LEFT JOIN no conservó la orden sin planchas.");` → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.

Línea 19: `Console.WriteLine($"INNER filas: {inner.Count} | LEFT filas: {left.Count}");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 20: `Console.WriteLine(_unidad.Ordenes.ObtenerSqlJoinExplicito());` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 21: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Línea 22: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<JoinsUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<JoinsUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.7 OK");
```

Explicación línea a línea de Program.cs 3.7

Línea 1: `using AceriaData.Application.UseCases;` → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.

Línea 2: `using AceriaData.ConsoleApp;` → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.

Línea 3: `using AceriaData.Infrastructure;` → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.

Línea 4: `using AceriaData.Infrastructure.Persistence;` → Importa `AceriaDbContext` para preparar la base de demostración desde el `composition root`.

Línea 5: `using Microsoft.EntityFrameworkCore;` → Importa las extensiones de EF Core utilizadas por `Migrate`, `ChangeTracker` y las operaciones de consulta del `checkpoint`.

Línea 6: `using Microsoft.Extensions.Configuration;` → Importa `ConfigurationBuilder` y las extensiones necesarias para cargar `appsettings.json` y variables de entorno.

Línea 7: `using Microsoft.Extensions.DependencyInjection;` → Importa `ServiceCollection`, `AddScoped`, `CreateScope` y `GetRequiredService` para componer el grafo de dependencias.

Línea 9: `var configuration = new ConfigurationBuilder()` → Inicia la construcción de la configuración de la aplicación.

Línea 10: `.SetBasePath(AppContext.BaseDirectory)` → Fija como base el directorio de ejecución para localizar `appsettings.json`.

Línea 11: `.AddJsonFile("appsettings.json", optional: false)` → Carga la configuración JSON obligatoria que contiene la cadena de conexión.

Línea 12: `.AddEnvironmentVariables()` → Permite sobrescribir configuración mediante variables de entorno.

Línea 13: `.Build();` → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer `ConnectionStrings:AceriaDB`.

Línea 15: `var connectionString = configuration.GetConnectionString("AceriaDB")` → Obtiene la cadena `AceriaDB` desde la configuración.

Línea 16: `?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");` → Detiene el arranque con un error explícito si no existe la cadena de conexión `AceriaDB`, evitando ejecutar el ejercicio con una configuración incompleta.

Línea 18: `var services = new ServiceCollection();` → Crea el contenedor de servicios del `composition root`.

Línea 19: `services.AddAceriaInfrastructure(connectionString);` → Registra `DbContext`, repositorio y unidad de trabajo de `Infrastructure`.

Línea 20: `services.AddScoped<JoinsUseCase>();` → Registra el caso de uso con ciclo de vida `Scoped` para compartir el mismo ámbito que `DbContext`.

Línea 22: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → Construye el proveedor de servicios y lo declara con `using` para liberar correctamente los servicios `IDisposable` al terminar el proceso.

Línea 23: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 24: `ValidateOnBuild = true,` → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.

Línea 25: `ValidateScopes = true` → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio `Scoped` consumido incorrectamente desde un `Singleton`.

Línea 26: `});` → Cierra las opciones y completa la llamada que construye el proveedor de servicios.

Línea 27: `using var scope = provider.CreateScope();` → Crea el ámbito `Scoped` que compartirá `AceriaDbContext`, `IUnidadDeTrabajo` y el caso de uso durante esta ejecución.

Línea 29: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` para preparar la base de demostración.

Línea 30: `context.Database.EnsureDeleted();` → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.

Línea 31: `context.Database.Migrate();` → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.

Línea 32: `DemoData.Seed(context);` → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.

Línea 33: `context.ChangeTracker.Clear();` → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.

Línea 35: `var useCase = scope.ServiceProvider.GetRequiredService<JoinsUseCase>();` → Resuelve el caso de uso concreto registrado para este checkpoint.

Línea 36: `useCase.Ejecutar();` → Ejecuta la demostración y sus aserciones E2E.

Línea 38: `Console.WriteLine("3.7 OK");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.7 OK
```

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: Un INNER JOIN elimina órdenes sin coincidencia. Si deben conservarse, se usa un patrón LEFT JOIN y propiedades anulables.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Código activable para reproducir el error controlado

Estos bloques ya existen comentados en el checkpoint. Actívalos de forma coordinada sólo para esta variante y restaura después el estado original.

Archivo: `src/AceriaData.Application/JoinsUseCase.cs`

Marcador: `ERROR CONTROLADO M03 3.7 - INNER JOIN PIERDE ORDEN SIN PLANCHAS`

```
// Demuestra el Paso 9: un INNER JOIN sólo conserva coincidencias.
var innerError = _unidad.Ordenes.ObtenerJoinOrdenesPlanchas();

if (innerError.Any(x => x.NumeroOrden == "OF-2024-0004"))
    throw new InvalidOperationException("Error controlado 3.7: el INNER JOIN conservó una orden sin planchas.");

Console.WriteLine("Error controlado 3.7 OK | INNER JOIN no contiene OF-2024-0004");
```

Comprobación esperada: INNER JOIN elimina la orden sin planchas.

Laboratorio adicional del punto 3.7

Diagnóstico técnico

`Join` modela coincidencias; el patrón de `left join` conserva la fila exterior y exige nulabilidad en la proyección interior. Con varias relaciones uno-a-muchos puede multiplicarse el número de filas: no son duplicados accidentales, sino el shape relacional.

Reto resuelto y verificación adicional

Compara `ObtenerJoinOrdenesPlanchas()` y `ObtenerLeftJoinOrdenesPlanchas()`: la orden sin planchas desaparece del INNER JOIN y permanece en el LEFT JOIN con datos relacionados nulos. Revisa la condición por `OrdenId` en el SQL.

Errores comunes revisados

Error	Causa	Solución
<code>NullReferenceException</code> en detalle	No se comprobó si el detalle es null	Usar operador ternario antes de proyectar
Se perdieron filas sin relación	En el patrón correlacionado se aplanó la colección sin <code>DefaultIfEmpty()</code>	Usar el patrón de <code>LEFT JOIN</code> con <code>DefaultIfEmpty()</code> cuando deban conservarse las entidades exteriores
Multiplicación de filas	Se cargaron colecciones hermanas con <code>JOIN</code> al mismo nivel	Evaluar proyección o <code>AsSplitQuery()</code> según el <i>shape</i>
<code>ThenInclude</code> sin <code>Include</code>	Se usó <code>ThenInclude</code> sin <code>Include</code> previo	Usar <code>Include</code> antes de <code>ThenInclude</code>
<code>GroupJoin</code> con forma no traducible	Se devolvió directamente la agrupación sin respetar un patrón soportado	Para un <code>LEFT JOIN</code> , aplanar inmediatamente con <code>DefaultIfEmpty()</code> y comprobar la traducción
Proyección de colección no validada	Se asumió que añadir <code>ToList()</code> determina la traducción o elimina N+1	Inspeccionar la traducción y reformular el <i>shape</i> ; <code>ToList()</code> no es una cura universal para N+1

Analogía operativa

Los joins en una acería son como las consultas que el jefe de planta hace al archivo central para combinar información de varias carpetas. En lugar de mirar la carpeta de órdenes, la carpeta de planchas y la carpeta de aleaciones por separado, el jefe pide un informe que combine las tres. El `Join` explícito es como indicar manualmente cómo se relacionan las carpetas; el patrón `GroupJoin + DefaultIfEmpty` permite conservar la carpeta principal aunque no exista una relacionada. Las propiedades de navegación permiten expresar relaciones sin escribir el join manualmente. En EF Core, la forma SQL concreta depende del patrón: puede aparecer un `JOIN`, una subconsulta u otra construcción, y la carga relacionada con `AsSplitQuery()` puede dividirse en varias sentencias coordinadas. Por eso el laboratorio inspecciona el SQL real en lugar de asumir una única forma.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Incluye órdenes sin planchas con LEFT JOIN y demuestra que OF-2024-0004 sigue presente.

Cómo activar el reto en el checkpoint

Estos bloques ya existen comentados en el checkpoint. Actívalos de forma coordinada sólo para esta variante y restaura después el estado original.

Archivo: src/AceriaData.Application/Interfaces.cs

Marcador: RETO M03 3.7 - PUERTO SQL LEFT JOIN

```
// Se activa coordinadamente con el repositorio y el caso de uso del reto.
string ObtenerSqlLeftJoinReto();
```

Archivo: src/AceriaData.Infrastructure/Repositories/Repositories.cs

Marcador: RETO M03 3.7 - SQL REAL DEL LEFT JOIN

```
// Usa el patrón GroupJoin + DefaultIfEmpty y permite inspeccionar la condición OrdenId.
public string ObtenerSqlLeftJoinReto() =>
    (from o in _context.OrdenesFabricacion
     join p in _context.PlanchasAcero on o.Id equals p.OrdenId into planchas
     from p in planchas.DefaultIfEmpty()
     orderby o.NumeroOrden
     select new
     {
         o.NumeroOrden,
         o.Cliente,
         Espesor = (double?)p.Espesor,
         Peso = (decimal?)p.Peso
     }).ToQueryString();
```

Archivo: src/AceriaData.Application/JoinsUseCase.cs

Marcador: RETO M03 3.7 - LEFT JOIN CONSERVA ORDEN SIN PLANCHAS

```
// Demuestra el Paso 10 y el laboratorio adicional comparando INNER y LEFT JOIN.
var innerReto = _unidad.Ordenes.ObtenerJoinOrdenesPlanchas();
var leftReto = _unidad.Ordenes.ObtenerLeftJoinOrdenesPlanchas();

if (innerReto.Count != 5 || leftReto.Count != 6)
    throw new InvalidOperationException("Reto 3.7: cardinalidad inesperada al comparar INNER y LEFT JOIN.");

if (innerReto.Any(x => x.NumeroOrden == "OF-2024-0004"))
    throw new InvalidOperationException("Reto 3.7: INNER JOIN no debe contener OF-2024-0004.");

var ordenSinPlanchas = leftReto.SingleOrDefault(x =>
    x.NumeroOrden == "OF-2024-0004" && x.Peso is null && x.Espesor is null);

if (ordenSinPlanchas is null)
    throw new InvalidOperationException("Reto 3.7: LEFT JOIN no conservó OF-2024-0004 con datos relacionados nulos.");

var sqlLeft = _unidad.Ordenes.ObtenerSqlLeftJoinReto();

Console.WriteLine($"Reto 3.7 OK | INNER: {innerReto.Count} | LEFT: {leftReto.Count} | Conservada: {ordenSinPlanchas.NumeroOrden}");
Console.WriteLine("RETO_LEFT_JOIN_SQL_INICIO");
Console.WriteLine(sqlLeft);
Console.WriteLine("RETO_LEFT_JOIN_SQL_FIN");
```

Comprobación esperada: LEFT JOIN conserva OF-2024-0004 y el SQL real contiene la semántica esperada.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.7 OK, el siguiente estado parte exactamente de esta solución y añade **Eager Loading con Include y ThenInclude**.

